

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 10: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper code lives in `starter_code.py` at the week root: the `FlowNetwork` class (with `add_edge`, an explicit residual-edge adjacency via `FlowEdge` objects and paired reverse edges, a reference `edmonds_karp`, and `copy/value` utilities), plus the random generators `generate_flow_network` and `generate_bipartite_graph`.

The 12 problems are grouped into three themes:

- **Flow Networks & Cuts** (Problems 1–3): the defining conditions of a flow, cut capacity, and constructing the residual graph.
- **Max-Flow Algorithms** (Problems 4–8): BFS augmenting paths, Edmonds-Karp, DFS Ford-Fulkerson, and extracting a minimum cut from a maximum flow.
- **Verification & Applications** (Problems 9–12): bipartite matching, multiple sources/sinks, edge-disjoint paths (Menger), and integrality – the reductions and structural guarantees that make flow so widely applicable.

1 Flow Networks & Cuts

Problem 1 – Flow Feasibility Checker

Implement `verify_flow(network, flow)`. Given a `FlowNetwork` and a candidate `flow` (a dict mapping each original edge (u, v) to a flow amount), check that the assignment is a valid flow: (1) every edge satisfies $0 \leq f(e) \leq c(e)$ (capacity constraints), and (2) flow is conserved at every node other than the source and sink (flow in = flow out). Return `(True, value)` when valid, where `value` is the net flow leaving the source; return `(False, 0.0)` otherwise. A flow that references an edge not present in the network is invalid.

Example. On the network with edges $s \rightarrow a(3)$, $s \rightarrow b(2)$, $a \rightarrow b(1)$, $a \rightarrow t(2)$, $b \rightarrow t(3)$, the assignment $\{sa:2, sb:2, at:2, bt:2\}$ is valid with value 4.

Problem 2 – Cut Capacity

Implement `cut_capacity(network, S)`. Given an s - t cut described by a set `S` (which must contain the source and not the sink), return the cut capacity: the sum of capacities of edges going *from* a node in `S` to a node not in `S`. Edges from `T` back into `S` do not contribute. Raise `ValueError` if `S` omits the source or contains the sink.

Example. On the network above, `cut_capacity(G, {"s"}) = 5` (edges $s \rightarrow a$ and $s \rightarrow b$).

Problem 3 – Build the Residual Graph for a Given Flow

Implement `build_residual_graph(network, flow)`. For each original edge (u, v) with capacity c and flow f : include a forward residual edge $u \rightarrow v$ with residual capacity $c - f$ when $c - f > 0$, and a backward residual edge $v \rightarrow u$ with residual capacity f when $f > 0$. Return an adjacency dict mapping each node to a *sorted* list of `(neighbour, residual_capacity)` pairs, omitting zero-capacity residual edges.

Hint. The backward edges are exactly what let an augmenting-path algorithm cancel previously routed flow.

2 Max-Flow Algorithms

Problem 4 – BFS for an Augmenting Path

Implement `bfs_augmenting_path(network)`. Using the current edge flows stored in the `FlowNetwork` (each `FlowEdge` exposes `residual_capacity()`), run a breadth-first search from the source and return a *shortest* (fewest-edges) augmenting path as a list of nodes `[s, ..., t]`, or `None` if the sink is unreachable in the residual graph. If the source equals the sink, return `[s]`.

Problem 5 – Edmonds-Karp Maximum Flow

Implement `edmonds_karp(network)`. Repeatedly find a shortest augmenting path by BFS; if none exists, stop. Otherwise compute the bottleneck (minimum residual capacity along the path) and push that much flow along it, updating forward and backward residual edges (use `FlowEdge.push`). Mutate the edge flows in place and return the max-flow value. This is Ford-Fulkerson with the BFS rule, giving the $O(VE^2)$ bound.

Example. `edmonds_karp(G) == 5.0` on the running example; on the unit-capacity “diamond” ($s \rightarrow a, s \rightarrow b, a \rightarrow b, a \rightarrow t, b \rightarrow t$) it returns `2.0`, demonstrating flow cancellation via the residual back-edges.

Problem 6 – Ford-Fulkerson with DFS Augmenting Paths

Implement `ford_fulkerson(network)` using *depth-first* search to find augmenting paths instead of BFS. Any augmenting path yields a correct max flow with integer capacities (DFS does not give the $O(VE^2)$ bound, but terminates on these small integer instances). Mutate the edge flows in place and return the value. It should agree with Edmonds-Karp on every instance.

Problem 7 – Verify Edmonds-Karp and Ford-Fulkerson Agree

Implement `verify_algorithms_agree(num_trials, n, seed)`: for each of `num_trials` random networks (`generate_flow_network(n, seed=seed + trial)`), compute the max flow with both your Edmonds-Karp (Problem 5) and your DFS Ford-Fulkerson (Problem 6), and return `True` iff the values match on every trial. Because the max-flow value is *unique* (it equals the min-cut), any two correct augmenting-path methods must agree. Use `network.copy()` so each algorithm runs on its own copy (the flow mutations are in place).

Problem 8 – Extract the Minimum Cut from a Max Flow

Implement `min_cut(network)`: compute a maximum flow, then let S be the set of nodes reachable from the source in the *final* residual graph (BFS along positive-residual edges). Return `(S, max_flow_value)`; the cut $(S, V \setminus S)$ is a minimum cut whose capacity equals the max-flow value (Max-Flow Min-Cut). Also implement `verify_max_flow_min_cut(num_trials, n, seed)` returning `True` iff, on every random network, `cut_capacity(G, S)` equals the max-flow value and the sink is not in S .

3 Verification & Applications

Problem 9 – Bipartite Matching via Max Flow

Implement `max_bipartite_matching(left, right, edges)` by reducing to max flow: add a super-source with a unit-capacity edge to each left vertex, a unit-capacity edge for each allowed pair (left u to right v), and a unit-capacity edge from each right vertex to a super-sink. The max-flow value equals the maximum matching size; by integrality the flow selects a valid set of matching edges. Label left vertices ("L", u) and right vertices ("R", v) so the two sides stay distinct.

Example. With `left=right=3` and edges $\{(0,0), (1,1), (2,2)\}$ the matching size is 3; with all of left pointing only at right vertex 0, it is 1.

Problem 10 – Multiple Sources/Sinks via Super-Source/Super-Sink

Implement `multi_source_sink_max_flow(nodes, edges, sources, sinks)` where each edge is a tuple $(u, v, \text{capacity})$. Introduce a super-source connected to every original source by an infinite-capacity edge, and a super-sink fed by every original sink via an infinite-capacity edge; the max flow between them equals the maximum total flow the sources can deliver to the sinks. Return that value.

Example. With sources $\{0,1\}$, sink $\{3\}$, and edges $0 \rightarrow 2(3), 1 \rightarrow 2(2), 2 \rightarrow 3(10)$, the answer is 5; lowering $2 \rightarrow 3$ to capacity 4 makes it 4.

Problem 11 – Edge-Disjoint Paths via Unit-Capacity Max Flow (Menger)

Implement `max_edge_disjoint_paths(nodes, directed_edges, s, t)`: the maximum number of pairwise edge-disjoint s - t paths. By Menger's theorem (the edge form), this equals the max s - t flow when *every* edge has capacity 1; with integral flow, each unit traces one edge-disjoint path. Return the count.

Example. On $s \rightarrow a \rightarrow t$ and $s \rightarrow b \rightarrow t$ the answer is 2; on a single chain $s \rightarrow a \rightarrow b \rightarrow t$ it is 1.

Problem 12 – Integrality of the Computed Flow

Implement `flow_is_integral(network)`: assuming a flow has been computed, return `True` iff every original forward edge carries an integer flow. Then implement `verify_integrality(num_trials, n, seed)`: for each random integer-capacity network, compute the max flow and check the flow is integral on every edge and the total value is itself an integer. Return `True` iff this holds on every trial.

Background. The Integrality Theorem guarantees this: starting from the all-zero flow with integer capacities, every augmentation pushes an integer bottleneck, so the flow stays integral throughout – which is exactly what makes the 0/1 matching and edge-disjoint-path reductions (Problems 9 and 11) work.

4 LeetCode Practice

After completing the problems above, practise this week's flow / min-cut / matching *modelling* skills on the following [LeetCode](#) problems. Note that *pure* max-flow problems are uncommon on LeetCode; the problems below are ones that are naturally modelled as flow, min-cut, or bipartite matching (and several are solved in practice by matching DP or BFS + min-cut intuition rather than a full Ford-Fulkerson implementation). Difficulty is LeetCode's own label.

- [Maximum Bipartite Matching → Maximum Number of Achievable Transfer Requests](#) – *Hard* – flow / enumeration.
- [Maximum Students Taking Exam](#) – *Hard* – matching / bitmask.
- [Minimum Cost to Connect Two Groups of Points](#) – *Hard* – assignment / min-cost matching.
- [Campus Bikes II](#) – *Medium* – assignment (min-cost matching).
- [Pizza With 3n Slices](#) – *Hard* – matching-style DP.
- [Escape the Spreading Fire](#) – *Hard* – BFS + min-cut intuition.